The Quantum Heist: Your Data Is Already Gone

Is Your Software Ready for the Quantum Era?

Paolo Patierno @ **Linux Day Napoli 2026**

About me



- Red Hat Senior Principal Software Engineer @ IBM
  - Working on Red Hat Streams for Apache Kafka
  - 100% open source
- Open-Source enthusiast
- Distributed systems (Apache Kafka & Strimzi)
- CNCF Ambassador

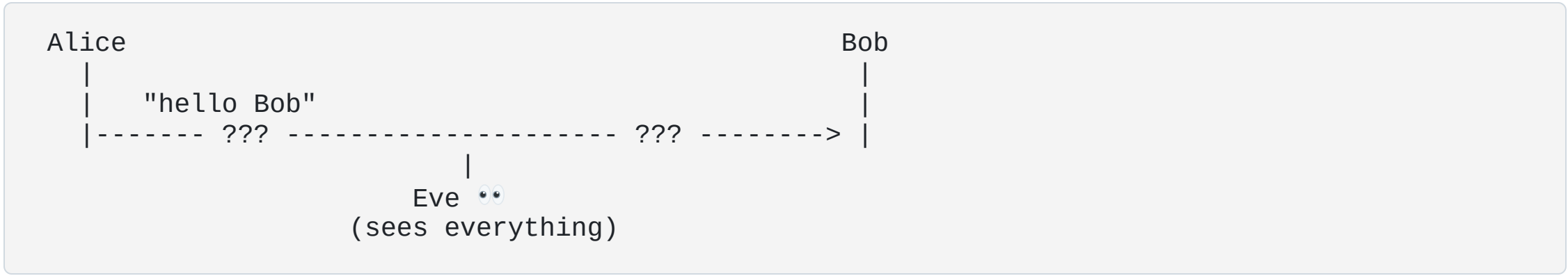
A guiding question

> *If quantum computers do not yet exist at the scale needed to break modern cryptography, why should I care about Post-Quantum Cryptography **today**?*

How does modern cryptography work?

The problem

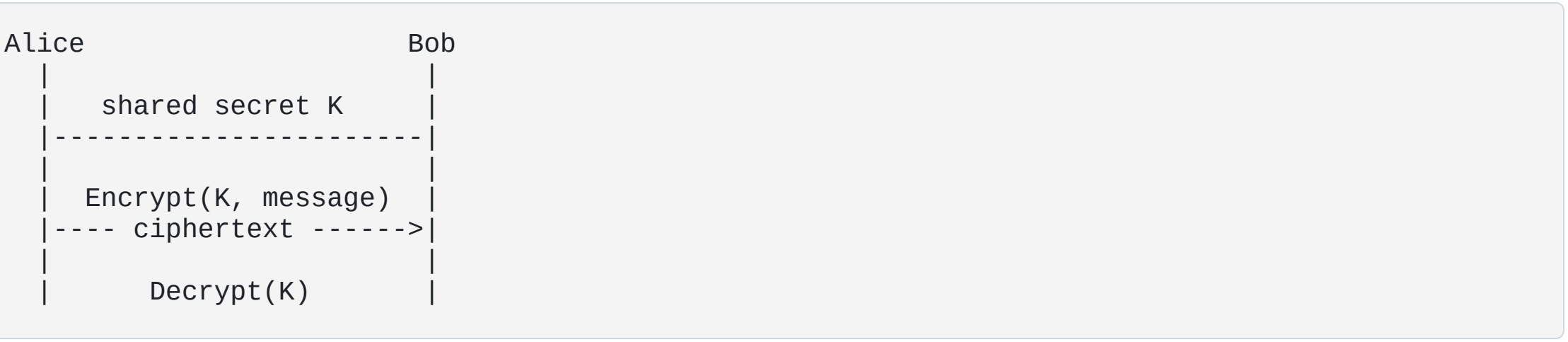
> *Alice wants to send a secret message to Bob over the Internet.*
> *Eve can see everything Alice sends.*
> *How can Alice and Bob communicate securely?*



Two fundamental approaches:

- **Symmetric cryptography**
- **Asymmetric cryptography**

Symmetric cryptography



Examples: **AES**, **ChaCha20**

> *Alice and Bob use the **same** secret key to encrypt and decrypt information.*

The key distribution problem

> *How do Alice and Bob securely obtain the shared secret **in the first place**?*

This is why asymmetric cryptography exists.

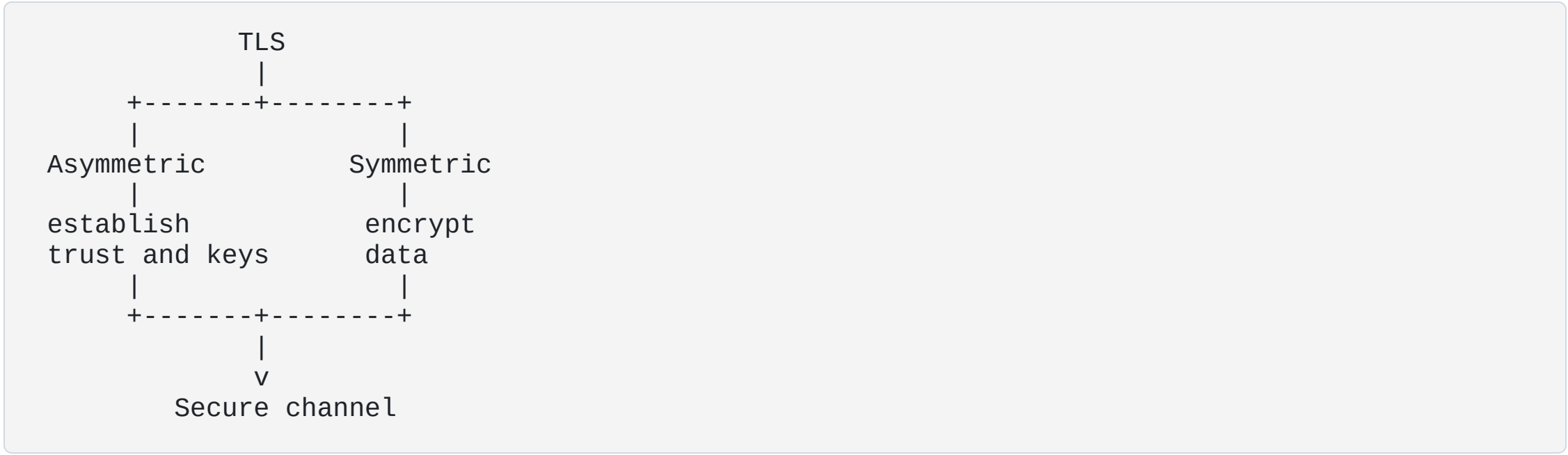
Asymmetric cryptography

```
        Bob
    +---------------+
    | Public key    |  <- everyone can know it
    | Private key   |  <- Bob keeps it secret
    +---------------+
```

Two important uses:

1. **Key establishment** — Alice and Bob establish a shared secret without already having one
2. **Digital signatures** — Bob proves a message came from him; Alice verifies with his public key

Bringing it together: TLS



> *This is how TLS works conceptually: public-key cryptography establishes trust and keys; symmetric cryptography encrypts the actual data efficiently.*

The mathematical foundation

> *Much of modern public-key cryptography relies on mathematical problems that are **easy to perform** but **computationally difficult to reverse**.*

```
RSA       -> integer factorization
DH / ECDH -> discrete logarithm
```

RSA — integer factorization

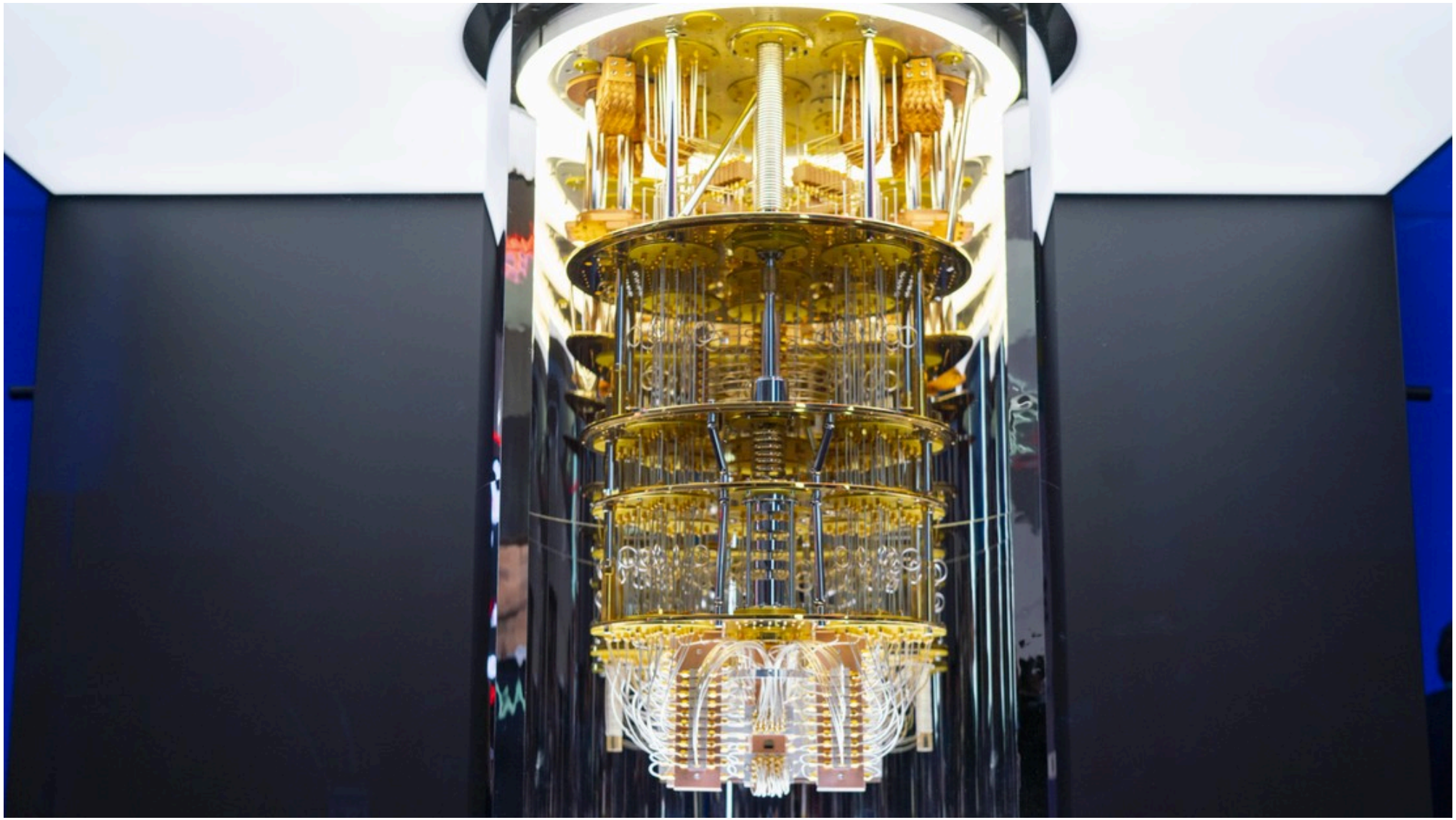
- ✅ Easy: multiply two primes → `p × q = N`
- ❌ Hard: given `N`, find back `p` and `q`

DH / ECDH — discrete logarithm

- ✅ Easy: compute `g^x mod p`
- ❌ Hard: given the result, `g` and `p` find `x`

What happens when the math becomes easy?

Do you know what this is?



Qubits and superposition

	State	Per operation
Classical bit	0 **or** 1	One value
Qubit	0 **and** 1 simultaneously	All values at once

```
Classical register (3 bits)    Quantum register (3 qubits)

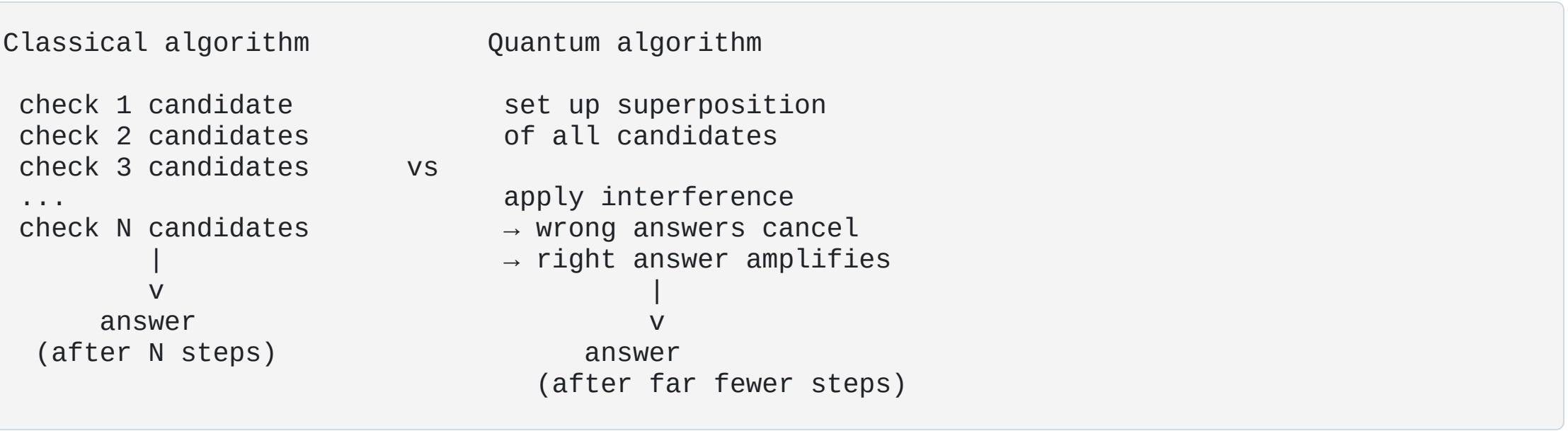
  stores one value               stores all 8 values
  at a time:  101                simultaneously:
                                 000, 001, 010, 011,
                                 100, 101, 110, 111
```

> *A quantum computer does not just compute faster — it computes **differently**.*

Why quantum algorithms are faster

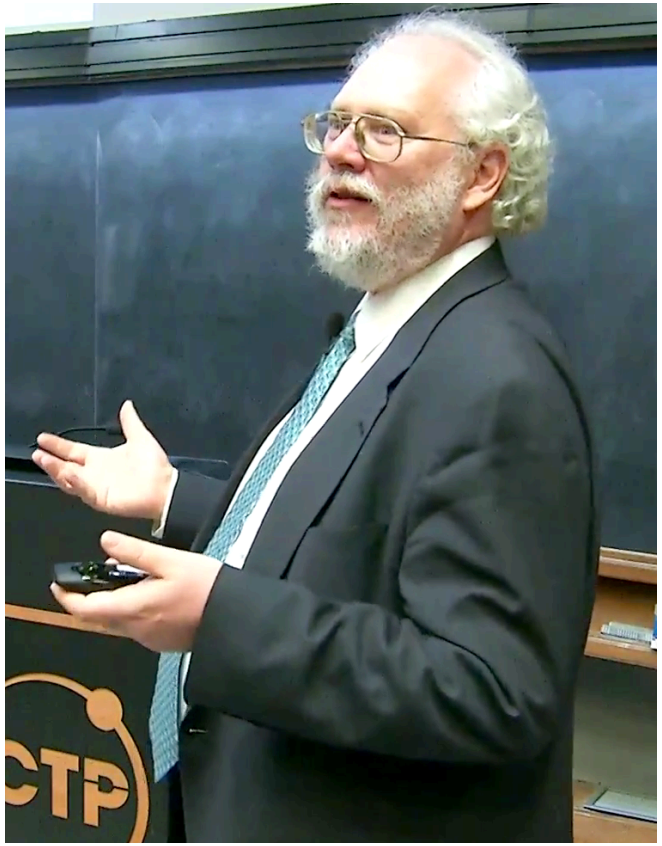
Three quantum properties working together:

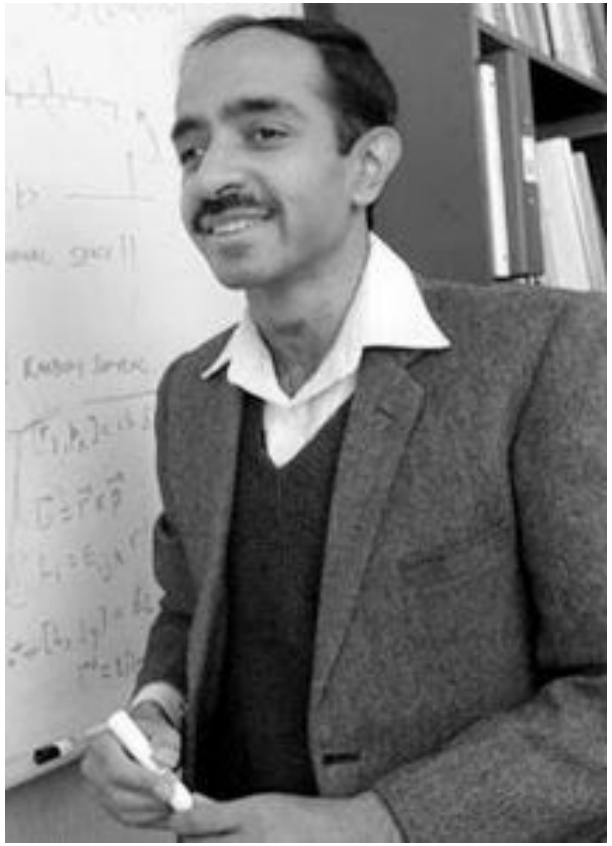
- **Superposition** — explore many inputs simultaneously
- **Entanglement** — qubits can be correlated so an operation on one instantly affects another
- **Interference** — amplify paths leading to the right answer, cancel paths leading to wrong ones



> *Not all problems benefit — only those with exploitable mathematical structure.*

Do you know who these people are?

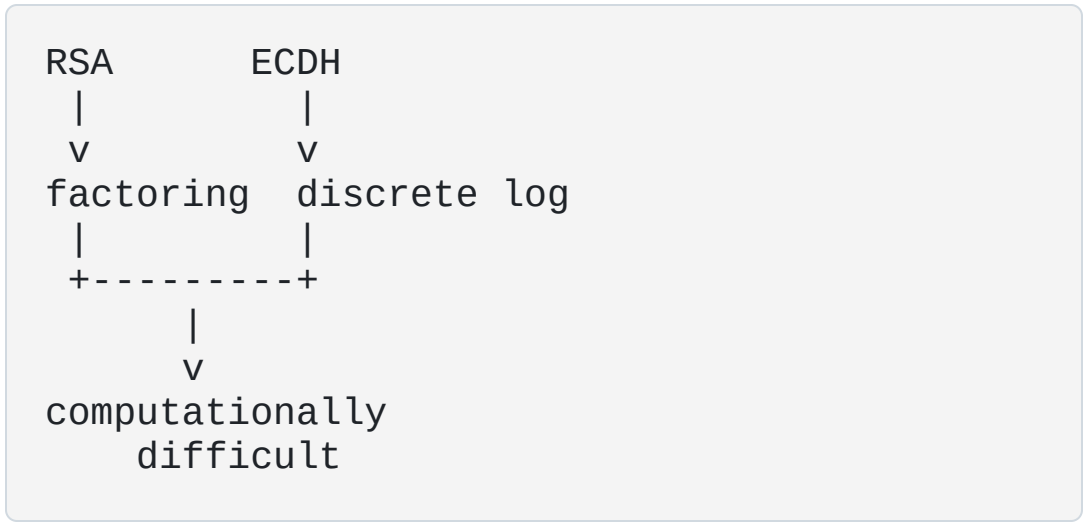




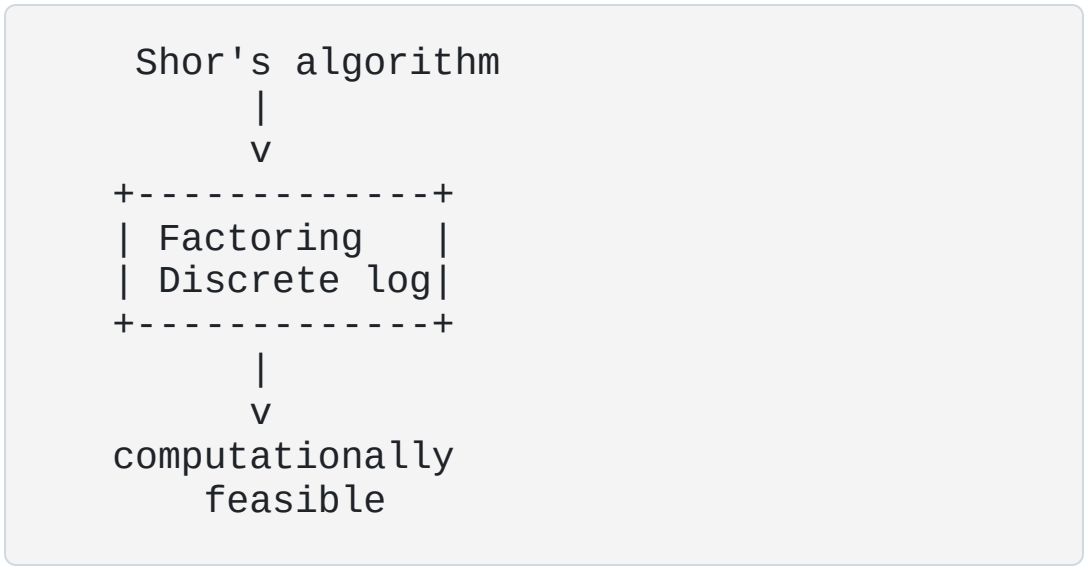
Enter Peter Shor

> *Shor's algorithm gives a sufficiently powerful quantum computer an efficient way to solve the problems behind **factoring** and **discrete logarithms**.*

Today



With a quantum computer



What becomes vulnerable?

- **RSA** — vulnerable
- **Diffie-Hellman** — vulnerable
- **ECDH** — vulnerable
- **ECDSA** — vulnerable

> *A quantum computer does not break the Internet by magic.*
> *It changes the **computational assumptions** on which some of our cryptography depends.*

What about AES? Enter Lov Grover

> *Does a quantum computer completely break AES?*

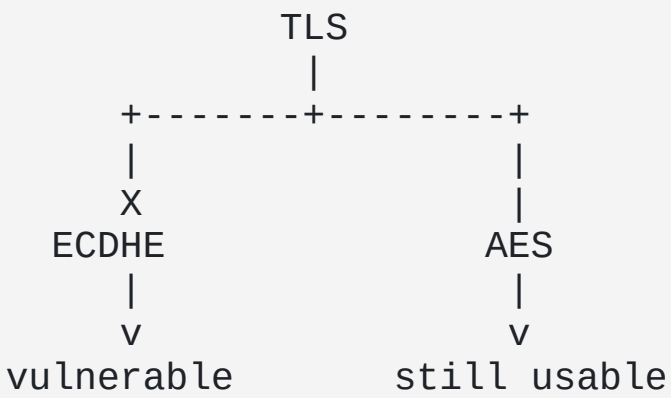
No.

Grover's algorithm provides a **quadratic speedup** for brute-force search:

```
Classical brute force       Quantum + Grover
AES-128                     AES-128
   |                            |
   v                            v
~2^128 possibilities        ~2^64 quantum operations
```

> *Quantum computers **fundamentally** threaten public-key cryptography.*
> *Symmetric cryptography is affected differently — it can be strengthened by using larger keys.*

TLS revisited

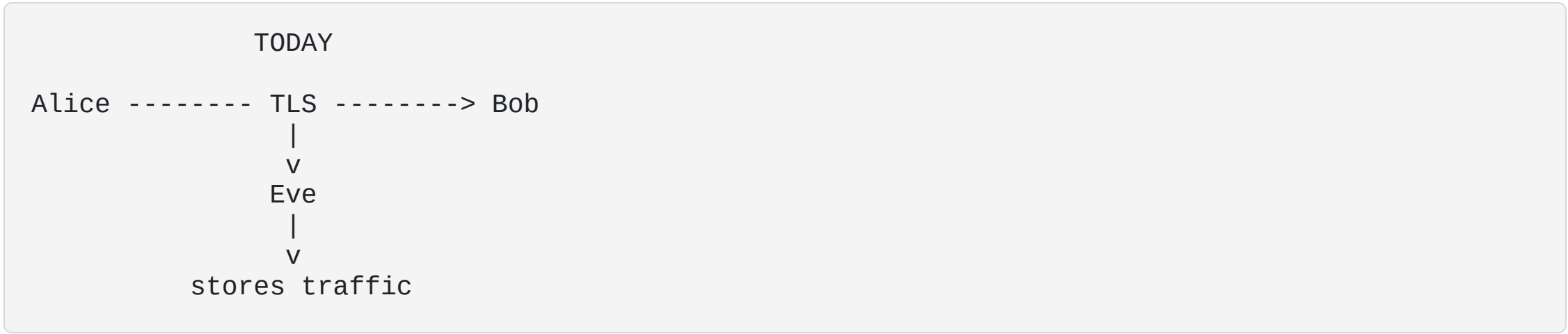


> *Quantum computers do not simply "break TLS".*
> *They threaten **some of the cryptographic building blocks** TLS relies on.*

Harvest Now, Decrypt Later

The apparent objection

> *OK, but quantum computers capable of breaking RSA and ECC do not exist today. So why should I care?*

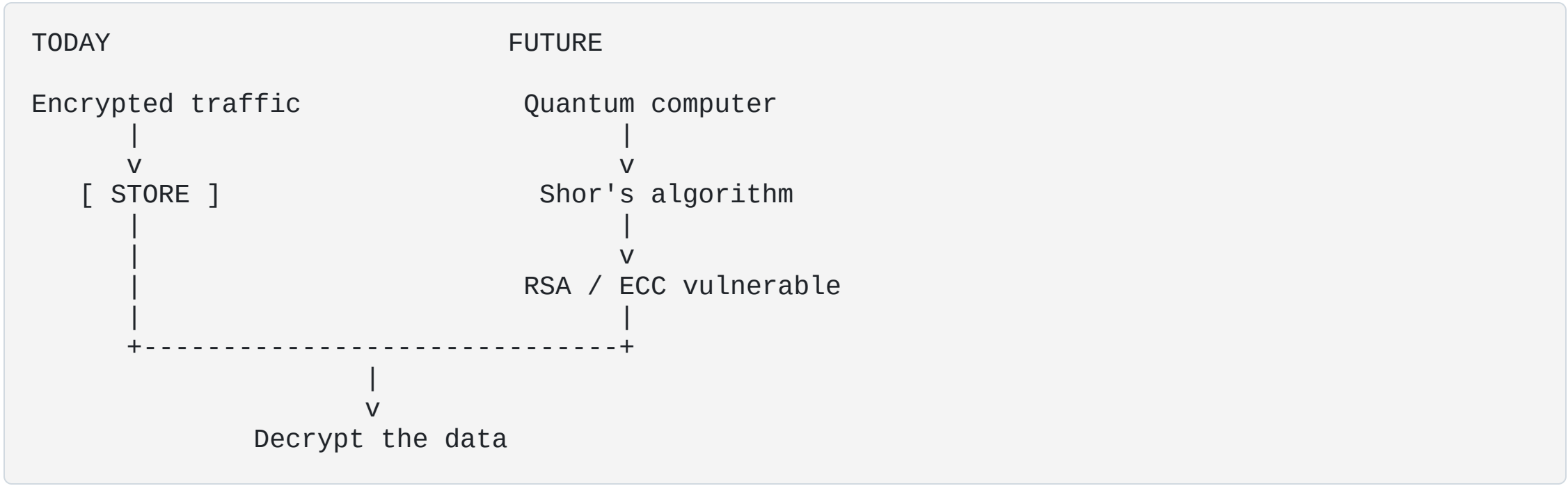


Eve does **not** need to decrypt the traffic today.

Eve can simply **record** it.

Eve is **patient** 😊

The future problem



Harvest Now, Decrypt Later — HNDL

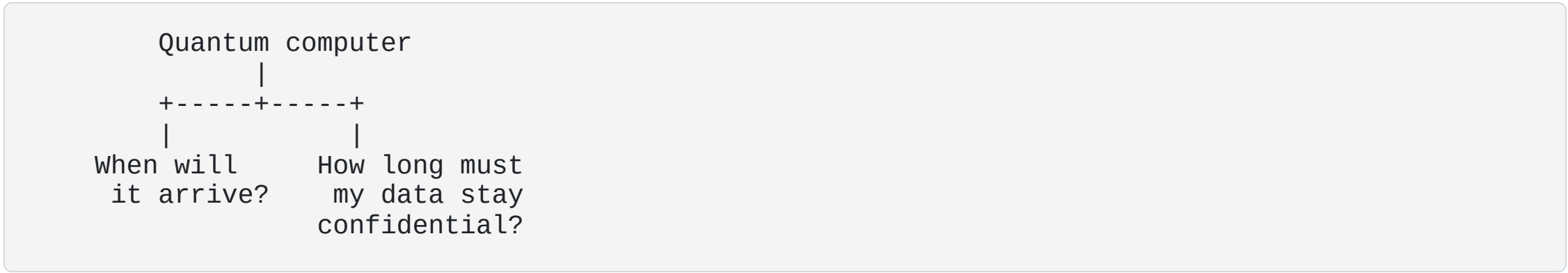
> *The confidentiality of data captured **today** may depend on the security of **tomorrow's** cryptography.*

What data is at risk?

Long-lived sensitive information:

- Confidential business communications
- Intellectual property
- Government information
- Personal information
- Long-lived secrets

Why this changes the migration timeline



> *We do not need to know **exactly** when a cryptographically relevant quantum computer will exist.*
> *If our data needs to remain confidential for 10, 20, or 30 years, we may need to **start the migration before** that computer exists.*

Post-Quantum Cryptography

What changes?

Classical cryptography

```
RSA / DH / ECC
      |
      v
Factoring / Discrete Log
      |
      v
     Shor
      |
      v
  Vulnerable
```

Post-Quantum cryptography

```
New mathematical problems
           |
           v
 believed difficult
 for both classical
 and quantum computers
```

> *We are not simply making RSA stronger.*
> *We are **changing the mathematical problems** on which our security depends.*

Why do Star Wars & Star Trek matter?





Module Lattice-based cryptography

> *Security relies on finding the **shortest vector in a high-dimensional lattice** — not factoring, not discrete logarithm.*

Lattice-based cryptography underpins both:

- **ML-KEM** *(formerly CRYSTALS-Kyber)* — key encapsulation
- **ML-DSA** *(formerly CRYSTALS-Dilithium)* — digital signatures

ML-KEM — Key Encapsulation Mechanism

> *How do Alice and Bob establish a shared secret?*

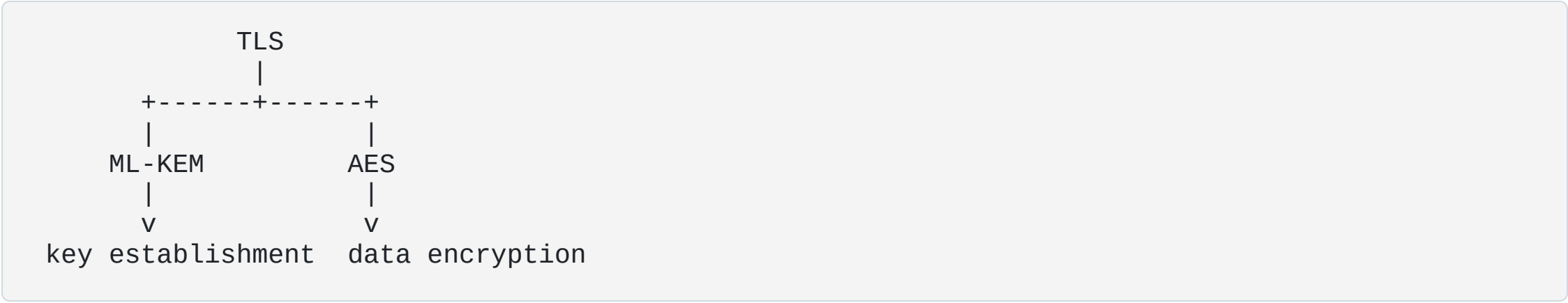
Classical

```
ECDH
 |
 v
shared secret
 |
 v
AES
```

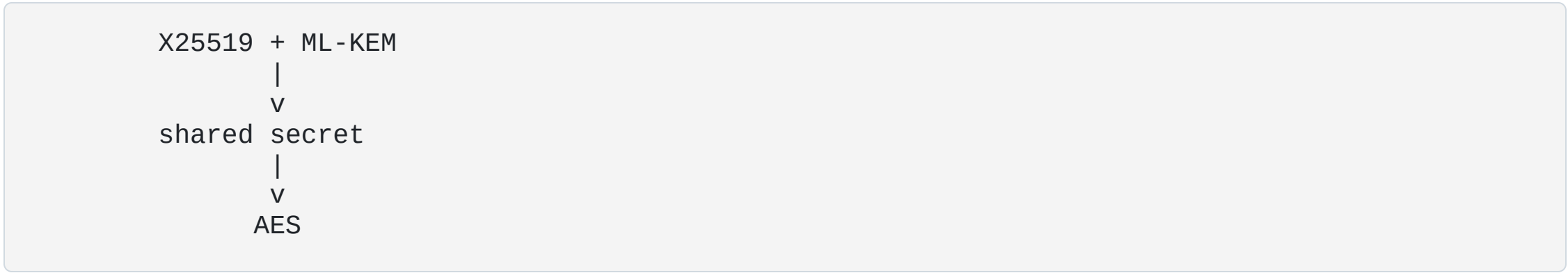
Post-Quantum

```
ML-KEM
   |
   v
shared secret
   |
   v
  AES
```

ML-KEM in TLS



Hybrid key establishment — transitioning safely



> *During the transition, we do not necessarily throw away classical cryptography overnight.*
> *We can combine classical and post-quantum mechanisms using properly designed **hybrid constructions**.*

ML-DSA — Digital Signature Algorithm

> *ML-KEM solves key establishment. But what about **authentication**?*

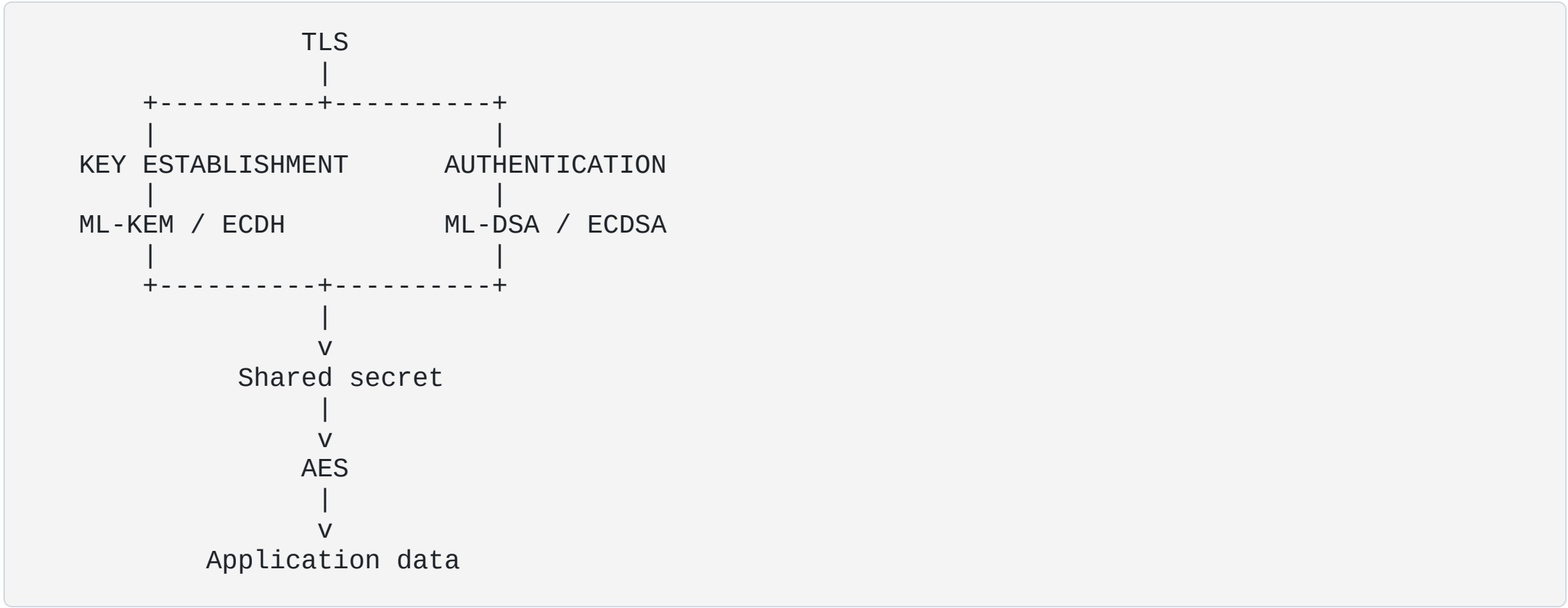
```
Server
   |
   | certificate / signature
   v
Client
   |
   v
Can I trust this server?
```

Classical: **RSA**, **ECDSA**, **EdDSA**

Post-Quantum: **ML-DSA**

> *ML-DSA — Module-Lattice-Based Digital Signature Algorithm*

The complete picture



Can I simply replace ECDH with ML-KEM?

> *If these algorithms are available, can I simply replace ECDH with ML-KEM and ECDSA with ML-DSA and call my application post-quantum ready?*

No.

Making your software PQC-ready

PQC is a software migration problem

> *PQC is not only a cryptography problem.*
> *It is a **software migration problem**.*

Step 1: Find where you use cryptography

```
Application
    |
    +-- TLS
    |
    +-- Certificates
    |
    +-- Authentication
    |
    +-- Database encryption
    |
    +-- Stored secrets
    |
    +-- Third-party libraries
```

> *You cannot migrate cryptography you have **not inventoried**.*

Cryptography is everywhere

Where are RSA, ECDSA, ECDH, AES hiding?

- Application code
- TLS configuration
- Certificates
- The JDK
- OpenSSL
- Reverse proxies
- Kubernetes components
- Cloud services
- Third-party dependencies

Step 2: Crypto agility

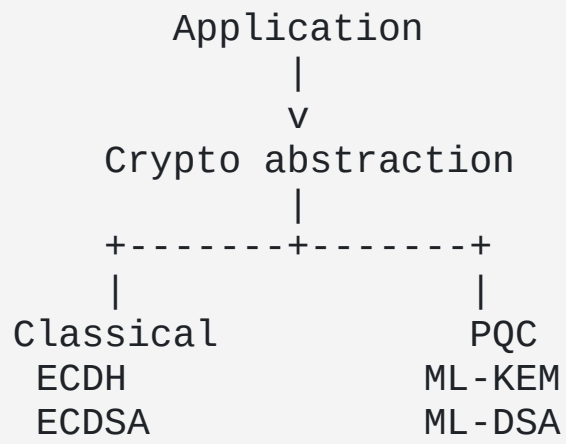
Software may assume hard-coded algorithms:

```
ECDH  = key exchange
ECDSA = signature
```

The question is not only whether these algorithms are secure **today**.

> *What happens when you need to **change** them?*

Design for crypto agility



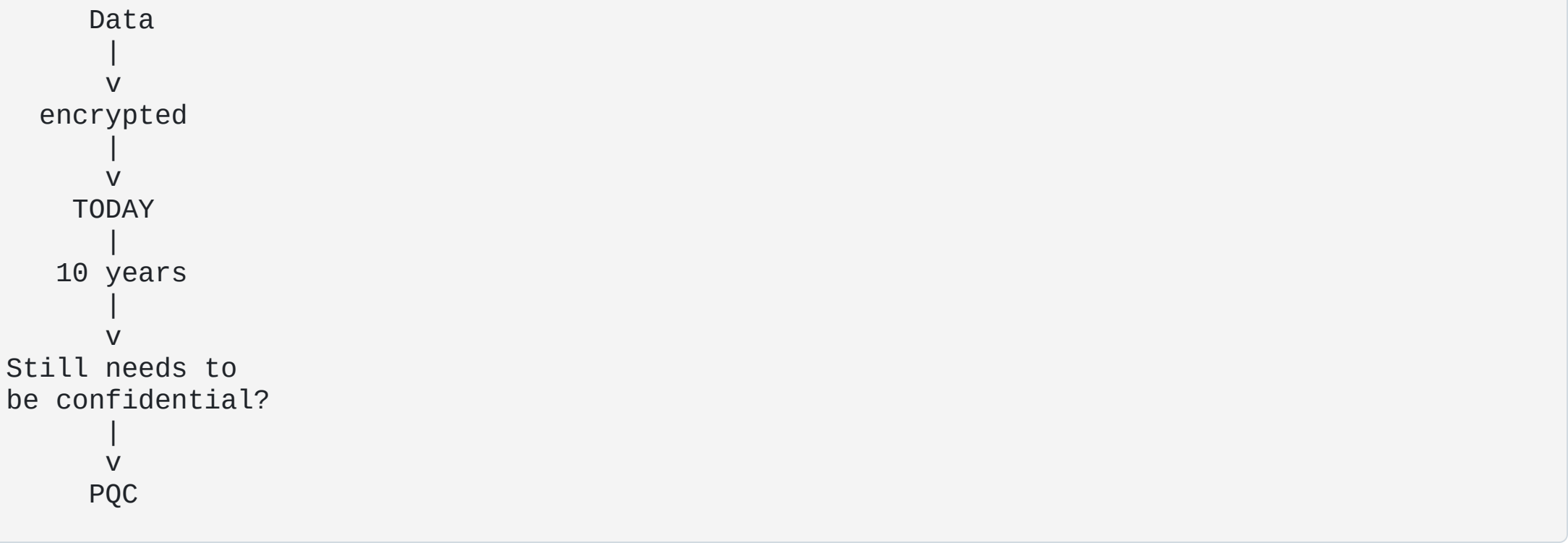
> ***Crypto agility*** *is the ability to change cryptographic algorithms and configurations without having to redesign the entire application.*

Step 3: Understand your dependencies

```
Your application
      |
      v
 Programming language / runtime
      |
      v
 TLS library
      |
      v
 JDK / OpenSSL / BoringSSL
      |
      v
 Operating system
```

PQC readiness depends on the **entire cryptographic stack**.

Step 4: Think about the lifetime of your data



> *How long does your data need to remain confidential?*

Putting it all together: the migration model

```
  1. INVENTORY
       |
       v
  2. IDENTIFY
quantum-vulnerable
     crypto
       |
       v
  3. BECOME
   CRYPTO-AGILE
       |
       v
  4. TEST
PQC / hybrid TLS
       |
       v
  5. MIGRATE
  where needed
```

Takeaways

Takeaway #1

> *Quantum computers threaten the mathematical assumptions behind today's **public-key cryptography**.*

```
RSA / DH / ECC
      |
      v
Shor's algorithm
      |
      v
 Quantum threat
```

Takeaway #2

> *The problem can exist **before** the quantum computer does.*

```
Harvest today
      |
      v
Decrypt tomorrow
```

Takeaway #3

> *PQC migration is a **software engineering problem**.*

```
Inventory
   |
   v
Crypto agility
   |
   v
Hybrid / PQC testing
   |
   v
Migration
```

You don't need a quantum computer to have a PQC problem.

> *The quantum computer is not necessarily the migration deadline.*
> ***The lifetime of your data is.***

Resources

- **NIST PQC Project** — Post-Quantum Cryptography
- **RFC 9846** — TLS 1.3
- **NIST FIPS 203** — ML-KEM specification
- **NIST FIPS 204** — ML-DSA specification
- **Post-Quantum TLS made easy in Java: Hybrid Key Exchange in action** — Paolo's blog post

Thank you

Linux Day Napoli 2026

Questions?